A FIELD MANUAL FOR SOLO BUILDERS

The 80% Wall.

Why AI-assisted builds stall between demo and production, and what it takes for one person to ship a real company.

MICAH JONES

CONTENTS

01 Why your build broke at 80% P. 3

02 The spec is the moat P. 12

03 The architecture you didn't draw P. 19

04 Deploy day P. 25

05 The security pre-flight P. 31

06 Stripe in production P. 37

07 Compliance, when it matters P. 44

08 The first ten users P. 50

09 The distribution loop P. 57

10 When to hand it off P. 63

01

Why your build broke at 80%

What happens in the context window when the AI starts undoing your features. The structural reason, not the vibes.

SUBJECT	CONTEXT WINDOWS · DRIFT	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER ONE OF TEN
TIME	A TEN-MINUTE READ	REV	2026.08

§ 01.1

The moment it turns

You asked for something small. Move a button. Fix a date format. The diff was four lines. It compiled. You shipped it.

Then you clicked around. Uploads are broken. Uploads. The feature you finished Tuesday. The one that already worked.

So you tell the AI to fix uploads. It fixes uploads. Now the date format is wrong again.

Somewhere around the third loop, everyone has the same thought: this thing has turned on me.

It has not. It is doing exactly what it was built to do, with exactly the information it has. The problem is structural, which is the good news. Structural problems can be engineered around, and engineering around this one is what separates the builds that ship from the graveyard of demos.

This chapter is the mechanism, in plain terms, and then five habits that stop the bleeding tonight. The other nine chapters build the cure.

FIELD NOTE

Everything in this manual comes from builds I shipped. Mostly Ordani: a HIPAA-compliant SaaS I built alone with Claude Code and Cursor, that hundreds of birth workers pay for. Same stack you're using. Same wall.

§ 01.2

What the AI actually remembers

Every AI coding tool, whatever the logo, works inside a **context window**.

CONTEXT WINDOW

> The fixed span of text the model can consider at one time: your instructions, the files it read, the diffs it wrote, the errors it saw. Everything outside the window does not exist for it. Not "harder to recall." Does not exist.

The window is not memory. It is a transcript with a hard length limit. In 2026 the big models hold somewhere between two hundred thousand and a million tokens, and a working session eats that fast: every file the tool opens lands in the transcript, along with every diff, every stack trace, every "here's what I found."

When the transcript fills, one of two things happens. The oldest content falls off the end, or the tool compresses it into a summary that keeps what looked important and quietly drops what looked incidental. "Looked" is the word doing all the damage.

Rough math: a token is about three-quarters of a word. Two hundred thousand tokens is a long novel. A session that reads files, writes diffs, and chases errors gets through a novel quickly.

And between sessions, almost nothing survives. Yes, your tool has a resume flag, and maybe a memory feature. Resume restores one conversation thread. Memory keeps what the tool guessed was important. Neither is a system you control. By default, tomorrow's session starts with three things: your codebase, your prompt, and no memory of how the codebase got this way.

§ 01.3

Features are more than their code

Here is the part almost nobody tells you. It is the crux of this whole manual.

A working feature is not just code. It is code plus the rules that make the code correct. "Uploads must stream to storage, never buffer, because a 500MB video crushes the server." "Dates format on the server, because the client's timezone lies." Engineers call a rule like this an invariant: something that must stay true no matter what changes around it. The code shows what it does. The reason it must be done that strange way lived in exactly one place: the conversation where you and the AI worked it out.

That conversation is gone.

So weeks later, a fresh session opens your upload file while doing something unrelated. It sees an oddly complicated streaming setup where a simple buffer would do. It has never heard of the 500MB video. Simplifying that code is the correct move given everything it can see. It is not sabotaging your feature. It is tidying code whose reason for existing is written down nowhere on earth.

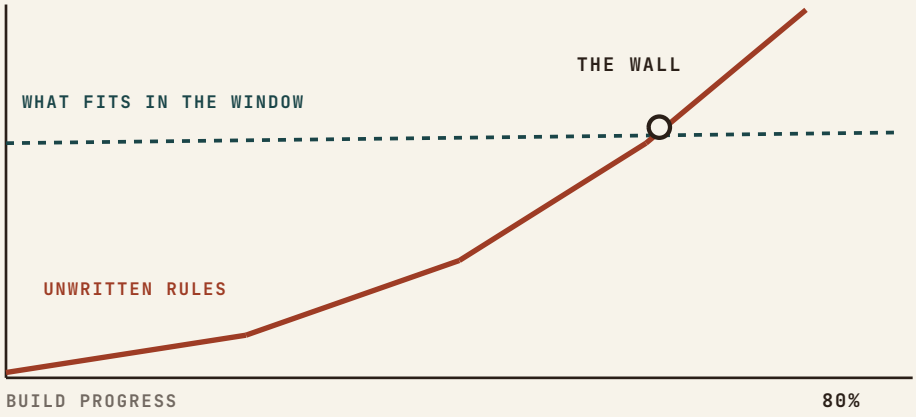
Code says what. Only the transcript knew why. And the transcript is gone.

Now multiply by every feature you have shipped. That is the true shape of your app at 80%: held together by dozens of invisible rules, inspected by a tool that can only see the code.

§ 01.4

Why it hits at 80% and not sooner

The wall is not one failure. It is two lines crossing.



Your codebase outgrows the window. In week one the whole project fits in context. The AI is briefly all-knowing, and it feels like magic. The magic is real, and it is temporary. A mid-sized app runs a few hundred files, several hundred thousand tokens, and past that the AI is no longer reading your codebase. It is sampling it.

Your unwritten rules pile up. Every feature that works adds constraints to the pile: the auth rule, the streaming rule, the timezone rule, the webhook-ordering rule. At 20% built you carry three of these in your head. At 80% you carry sixty, they interact, and not one is written down.

Your sessions multiply. By 80% you are dozens of sessions deep. Each started amnesiac. Each rebuilt its own partial picture of the app from whatever files it happened to open. Session forty's mental model disagrees with session twelve's, and both wrote code you are still running.

The wall is the point where the invisible rules outnumber what fits in the window. It is not a talent problem. It is arithmetic, and arithmetic can be beaten with a system.

§ 01.5

The regression loop

Cross the wall without knowing it, and the same spiral catches nearly everyone:

1. You ask for change B. The AI delivers it, quietly violating rule A, which was nowhere in its view.
2. You spot A broken and ask for a fix. The fix trips rule C, or re-breaks B.
3. The fixes keep compiling, so you stop reading the diffs. Drift now compounds silently.
4. Frustration peaks, and the thought arrives: maybe I should just have it rebuild the whole thing clean.

> The rewrite is the wall wearing a costume. A rebuild trades bugs you know for bugs you do not, and resets your pile of hard-won rules to zero. That pile was the only thing you had earned. Rebuilds stall at their own 60%, for exactly the reasons the original stalled at 80.

FIELD NOTE

The break rarely announces itself. The first symptom is usually a regression in something you had stopped watching, days after the session that caused it.

§ 01.6

Two entries from my build log

I hit this wall in the same month I am writing this, on my own consulting site, with a long engineering career behind me. The wall does not care about your resume. Both entries below are real, with dates.

FROM THE BUILD LOG

ENTRY · 2026-08-11

The same bug, twice

My site's framework has a quirk: in certain conditions the renderer eats the space after bold text, so "$20M+ in client revenue" shipped to production as "$20M+in client revenue." A session found it, fixed it, and wrote the lesson in the project's docs. Case closed.

Weeks later, a different session reintroduced the exact same bug in new copy. It had never read the lesson. Writing it down was not enough, because prose only works on whoever reads it, and the next session reads nothing you don't put in front of it.

What finally stuck was a one-line check in the repo, run before every release. Simplified, it looks like this:

```
grep -rn "+in client" app/ && echo "SPACE BUG IS BACK" && exit 1
```

In plain English: search every file for the broken pattern, and if it appears anywhere, shout and refuse to ship. The bug never shipped again. Rules a machine can run beat rules a reader must remember. That idea repeats through every chapter of this manual.

FROM THE BUILD LOG

ENTRY · 2026-08-31

The demo that lied for weeks

That same site had three lead forms: contact, a sample-chapter signup, a beta waitlist. All three worked flawlessly in the demo. In production, for weeks, every submission fell into a server log nobody reads, because one environment variable, the email key, was never installed on the live host.

No error. No bounce. The page told every visitor "Got it."

I found out this morning, only because I tested a new feature end to end on the live site and that test failed loudly enough to make me look. Two lessons, each with its own chapter later: production is a different machine than the demo. And "it works" is a claim about the path you actually tested, never about the code you wrote.

§ 01.7

What does not fix it

Four false exits. I have paid for all four.

A smarter model. Smarter per glance, same amnesia. Drift is not caused by low intelligence. It is caused by what is out of view.

A bigger window. A million tokens digests a bigger codebase. It does not conjure the sixty reasons-why that were never written anywhere a window could hold.

Telling it to remember. "Always keep uploads streaming" pins the rule to one transcript. Politeness is not persistence. The next session never heard you.

Pasting everything, every time. Stuffing the repo into each prompt burns the window on what while still holding zero why, and crowds out the room the model needs to think.

§ 01.8

What works: memory the window cannot lose

The move that changes everything is small: stop treating the conversation as storage.

Anything that must survive the session goes in a file, in the repo, because files are the only thing every future session is guaranteed to see. A rules file has a second advantage: when the tool compresses your conversation, the transcript gets summarized, but the rules file gets re-read, whole, every time. Your role quietly changes at the wall. You stop being the person who asks for features, and become the keeper of the memory the tool does not have. That is not a demotion. It is the actual job, and the people who accept it are the ones who ship.

PRE-FLIGHT · FIVE HABITS — RUN TONIGHT

- [ ] **Write the invariant list.** One file in the repo root, named whatever your tool reads on its own (see the file card below). One line per rule-with-a-reason: “Uploads stream, never buffer: 500MB videos kill the function.” Thirty minutes, tonight.

- [ ] **Point every session at it first.** “Read the invariants file before you touch anything.” Ten seconds that stands in for every session that came before this one.

- [ ] **Read every diff before accepting.** The moment you stop reading is the moment drift starts compounding. If a diff touches a file your change had no business touching, stop and ask why.

- [ ] **One change per commit, committed when it works.** Small commits are restore points. Rolling back beats an apology prompt, every single time.

- [ ] **Caught the AI breaking a rule? Write the rule down, same day.** Fix the code, add the rule to the file, and where you can, turn it into a check a machine runs. Nothing in this manual pays back more per minute.

So you don’t have to guess what habit one produces, here is the shape of the file, three rules in. Yours will grow past twenty.

CLAUDE.md — invariants

```
# Rules that must survive every session. Read
before any change.
• Uploads STREAM to storage, never buffer. 500MB
  videos kill the server.
• Dates format on the SERVER. The client's timezone
  lies.
• Never hand-edit files in /generated. Regenerate
  them or nothing.
```

These five stop the bleeding. They are triage, not the cure. The cure is a single page the AI re-reads at the start of every session, carrying your architecture, your constraints, and your definition of done. Building that page is Chapter 2, and it is the reason this manual exists.

Tool note: Claude Code reads CLAUDE.md automatically. Cursor reads the .mdc rule files in .cursor/rules. Codex and several others read AGENTS.md. Find the file your tool reads first. Same idea, same payoff.

The wall is not proof you can’t do this. It is the point where the tool’s memory ran out and yours has to take over: on paper, in the repo, where every future session finds it.

02

The spec is the moat

The one page the AI keeps re-reading. Why drift, not bugs, is what kills your build. Template included.

SUBJECT	SPECS · DRIFT CONTROL	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER TWO OF TEN
TIME	A SEVEN-MINUTE READ	REV	2026.08

§ 02.1

The killer isn’t bugs

Bugs are loud. They throw errors, fail the build, break the demo in front of your friend. You find bugs because bugs want to be found.

What kills builds is quieter. Somewhere around the fortieth session, you open your app the way a stranger would, and you feel it: everything works, and this is not the product you meant to build. Settings has eleven screens. There’s a dashboard you never asked for, grown from a “while I’m here” suggestion you approved on a tired Tuesday. The one flow that had to be effortless takes four taps.

Nothing is broken. That is exactly the problem.

DRIFT

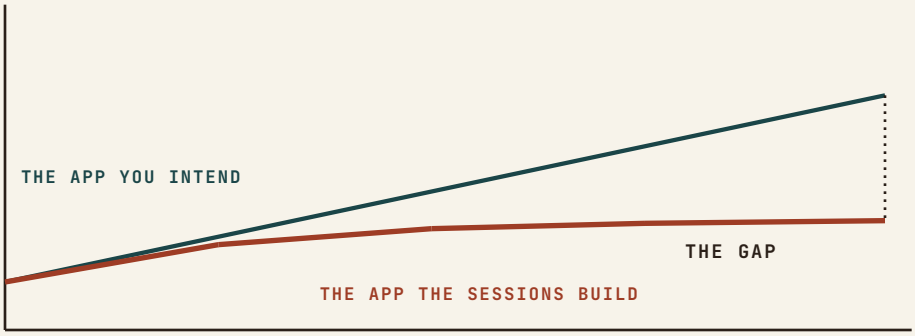
> The accumulating gap between the product you intend and the product the sessions are actually building. Not wrong code. Wrong direction, shipped correctly, one reasonable-looking diff at a time.

Chapter one gave you the invariant list, and that file is defense: it stops a session from undoing what already works. It does nothing to steer what gets built next. Steering is this chapter.

§ 02.2

Where drift comes from

Recall the mechanism. Every session starts by guessing what you are building. It reads your ask, samples whatever files it happens to open, and re-derives the product’s intent from the evidence. Forty sessions means forty independent guesses at what you want. Each one lands close. Close, compounded forty times, is far.



The intent lives in exactly one place the tool cannot read: your head. Every feature request you type is a translation from that head, made under deadline, at night, in fragments. The AI fills every gap in the translation with the most statistically ordinary choice. Your product drifts toward the average of every app it has ever seen, because that is what gap-filling means.

FIELD NOTE

Drift is why every half-finished AI build looks eerily alike: the same dashboard, the same card grid, the same settings sprawl. The average is what fills the gaps you left.

§ 02.3

One page, in the repo

The fix is the same move as chapter one, pointed at direction instead of correctness: put the intent where every session reads it before it reads anything else.

Not a product requirements document. Thirty pages is where intent goes to be skimmed. A spec that works in an AI-tool loop has one hard property: it fits in one page, so it gets read whole, every session, without eating the window. One page also forces you to decide what actually matters, which is most of the value before the tool ever sees it.

Six sections. Each is one to six lines.

WHAT. One sentence: the product, who it’s for, and the behavior that means it’s winning.

NOT. What this product refuses to be or do. The anti-scope.

SHAPE. The architecture in six lines or fewer: where data lives, who’s allowed to see what, what talks to what.

RULES. Your top invariants, or a pointer to the invariants file.

NOW. The current milestone, one line, with what “done” means for it.

LATER. The parking lot. Ideas go here to wait, instead of leaking into scope.

The strange one is NOT, and it does the most work. Drift rarely enters as a bad idea; it enters as a good idea that belongs to a different product. The AI is an enthusiastic yes-machine, and every “while we’re at it, should I add…” is a fork in the road. With a NOT section, that fork is not a judgment call at midnight. It is a lookup.

SPEC.md

```
# WHAT
Booking and payments for independent personal
trainers. One trainer, their clients, their
calendar. Winning = a client books and pays in
under a minute on a phone.
# NOT
Not a marketplace. Not multi-trainer gyms. No
social feed, no chat, no meal plans. No admin
dashboard beyond earnings and the calendar.
# SHAPE
Next.js on Vercel. Postgres w/ row-level security:
trainers see only their own clients. Stripe for
payments, webhooks own booking state. Twilio for
reminders. No other third parties.
# RULES
See CLAUDE.md invariants. Top three: bookings are
never double-writable; money state comes only from
Stripe webhooks; phone numbers are stored E.164 or
not at all.
# NOW
Milestone: first paying trainer. Done = she books
10 real sessions in a week without texting me for
help.
# LATER
Packages/subscriptions. Waitlists. Google Calendar
sync. Referrals.
```

E.164, from the RULES line below: the international phone format. A plus sign, country code, then digits: +14155551212. Store one format, or search and duplicate-matching quietly break later.

Steal the shape, not the contents. Yours will be shorter in some sections and longer in NOT than you expect.

§ 02.4

Why this is the moat

Everyone has the same tools now. Same models, same editors, same starter templates, and the demos all look the same because of it. What separates the builds that become

products is not the tool. It is how little the builder lets the tool guess.

The spec is the one input the AI cannot generate, because it is the one thing only you know: what you actually want, and what you refuse to ship. Written down, it compounds. Every session starts aligned instead of guessing. Every proposal gets checked against NOT at the moment it's made, which is the cheap moment. Drift caught at proposal costs one sentence. Drift caught at regression costs a weekend, and chapter one showed you the weekend.

Same tools in everyone's hands. The page that says what you want is the only part they can't copy.

Two entries from my build log

Both of these are mine, both dated. The first cost six weeks. The second cost a week, and I wrote this manual partly because of it.

FROM THE BUILD LOG — ENTRY · 2026-05-19

Six weeks toward a reference nobody locked

On Ordani, I spent six weeks polishing the interface toward a quality bar that existed only as a feeling. Every session, the AI and I made real improvements against an imagined reference. The post-mortem line still stings: the work was optimized toward a direction that was never agreed, not even with myself.

The fix was not better design work. It was one page, written before the next round, that said what good means here: the reference products, the feel, and the anti-patterns to refuse. The page took an evening. The six weeks did not come back.

FIELD NOTE

The tools themselves are converging on this idea: plan modes, rules files, project instructions. A written plan the model critiques before code is the same move wearing a feature name. The habit transfers to every tool you'll ever use.

§ 02.5

FROM THE BUILD LOG

ENTRY · 2026-08

Four redesigns in one week

My own consulting site, the redesign that produced the site you found this manual on. Four full design rounds in a single week, every one rejected, some within minutes. The instruction I had given: “make it better.” The AI obliged, four different ways, toward four different averages.

Round five started with a locked sentence: nicer than what exists, no cheap gimmicks, photos of real work, keep what already worked. A one-line WHAT and a three-item NOT. It shipped in two passes, and I love it. Same tool. Same week. The page was the difference.

§ 02.6

The ritual

The spec only works as a habit loop. Five moves:

PRE-FLIGHT · THE SPEC RITUAL — RUN TONIGHT

- [ ] **Write SPEC.md tonight.** Six sections, one page, hard cap. If it spills past a page, NOT and LATER are where the excess goes.
- [ ] **Open every session with it.** "Read SPEC.md and the invariants file, then the task." Ten seconds. Direction and defense, loaded before any code.
- [ ] **Frame asks against NOW.** "Per the NOW milestone, build X" beats "build X." It tells the tool what to optimize for and, just as useful, what not to gold-plate.
- [ ] **Treat every "should I also…" as a NOT lookup.** If it's in NOT, the answer is no and costs nothing. If it's genuinely new, it goes to LATER, deliberately, not into the diff.
- [ ] **Change the spec only in its own commit.** The spec is allowed to evolve; it is not allowed to drift. A spec edit that rides along inside a feature diff is drift with paperwork.

One page, six sections, read at every session start, guarded by a five-move ritual. That is the moat. The next chapter draws the one diagram the SHAPE section summarizes: the architecture you didn't draw, and where AI tools quietly cut corners in it.

03

The architecture you didn't draw

The single diagram every solo build needs. Auth, data, storage, third parties, and where AI tools quietly cut corners.

SUBJECT	ARCHITECTURE · TRUST LINES	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER THREE OF TEN
TIME	A SIX-MINUTE READ	REV	2026.08

§ 03.1

The napkin test

Here is a test that takes ten seconds. Can you draw your app on a napkin? Not the screens. The machinery: where the data lives, what talks to what, and who is allowed to ask for what.

If you can't, you are not behind. You are normal. AI tools build bottom-up: file by file, feature by feature, each one locally sensible. Nobody ever asked you for the top-down view, so it doesn't exist. The app works and you cannot draw it.

That gap stays invisible until the day it doesn't. The first security question from a real customer. The first "can I export my data?" The first bug that lives between two services instead of inside one file. Every one of those is answered with the drawing you don't have.

This chapter is that drawing: five boxes, universal to almost every solo SaaS, plus the specific corners AI tools cut inside each one. It is also your spec's SHAPE section (chapter 2), drawn instead of written.

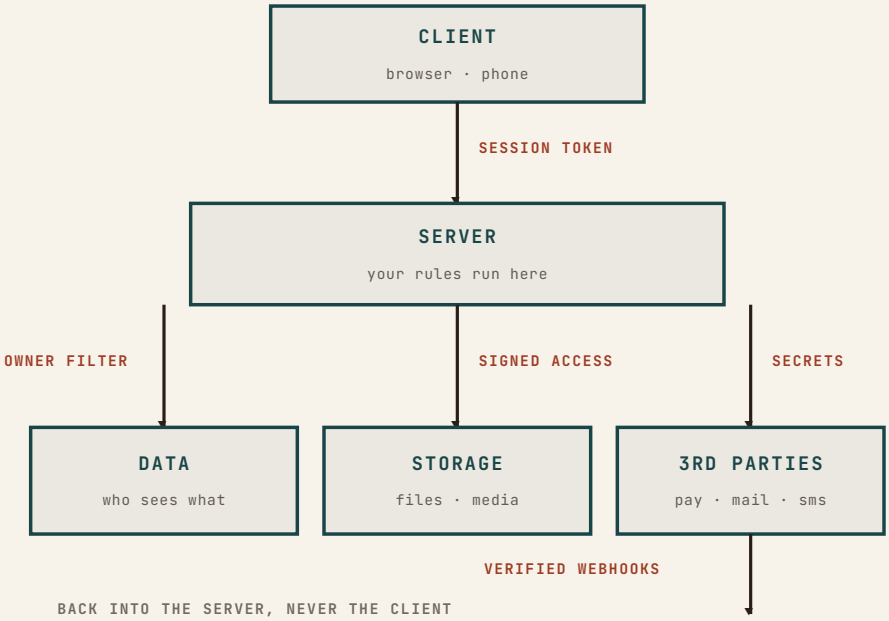
FIELD NOTE

The napkin is also the first thing a buyer, an investor, or a security reviewer asks for. "Walk me through the architecture" is the adult version of "does it work?"

§ 03.2

The five boxes

Nearly every app you will build alone reduces to this map:



Client. The browser or phone. Everything the user touches, and everything an attacker touches too, because the client runs on hardware you do not control.

Server. Your functions, your API routes. The only place where a rule is actually a rule.

Data. The database, and the single most important sentence in your architecture: who is allowed to see which rows.

Storage. Files. Uploads, exports, images. A separate box because it fails separately, and publicly.

Third parties. Payments, email, texts, calendars. Money and messages leave your app here, and truth about money comes back here.

The arrows matter more than the boxes. Every arrow is a trust line, and every trust line needs an answer to one question: what stops the wrong person from using this? Five boxes, five or six arrows, one lock per arrow. That is the whole discipline.

Written down it is a short file, and it ships in the companion pack as `ARCHITECTURE.md`, filled in for the trainer app from chapter two.

§ 03.3

Where the AI cuts corners, box by box

None of what follows is the AI being careless. It is the AI being plausible. Cut corners compile fine and demo fine. They fail only in production, against strangers.

Client: it will trust the browser

The most common corner. Validation that lives only in the form. Prices, quantities, and role flags sent from the client and believed by the server. A “this button is hidden for non-admins” standing in for an actual permission check.

The rule: the client is a suggestion box. Anything it enforces is decoration until the server enforces it again. When you ask the AI for a feature, ask for the server-side check by name, or you will often get the decoration alone.

Server: it will assume your laptop

Code that works locally leans on things production doesn’t have: environment variables that exist only on your machine, file paths, a database seeded by hand. The corner is invisible

because the demo is your laptop. Chapter one's dead-forms story lives in this box, and chapter four is entirely about it.

Data: it will forget who owns the row

Ask an AI for "a function that fetches invoices" and you will frequently get exactly that: all the invoices. The ownership filter, `where user = current_user`, is the single most safety-critical line in your app, and it is precisely the kind of line that drops out when a later session rewrites a query it half-understands.

The strong version of the fix does not trust your queries at all: ownership enforced in the database itself, so a query without the filter returns nothing instead of everything. In Postgres this is row-level security, and it is the difference between "every query must remember the rule" and "the rule is physics."

Storage: it will make the bucket public

Uploads are the classic. The AI wires file upload in one pass, it works, everyone moves on, and the bucket is world-readable because that was the path of least resistance. Photos, PDFs, exports: if the URL works in an incognito window, it works for everyone on earth.

The rule: storage is private by default, and the server hands out temporary signed links. "Can a logged-out stranger open this file's URL?" is a ten-second test. Run it on your own app tonight.

Third parties: it will paste the key where it worked

Two corners here. The first is secrets: API keys belong in environment variables, set on the host, never in code, never in config files the AI writes. The second is truth: when money moves, the payment provider's signed webhook is the fact, and whatever your client says happened is not. Chapter 6 is that story in full.

FIELD NOTE

Ordani runs this way: a birth worker sees her clients and nobody else's, enforced in the database, not in my query discipline. That design has been through two outside security reviews. Chapter 5 builds it step by step.

FROM THE BUILD LOG

ENTRY · 2026-08

The token that was everywhere

Auditing a client project, I searched the repo for anything shaped like a credential. One live API token appeared twelve times, pasted into tool-configuration files, each copy written by an AI session that was quietly being helpful: the command needed the token, the tool saved the command, and nobody looked.

No attacker was involved. No one made a bad decision. Twelve copies of a live key accumulated one convenient moment at a time, in files nobody reads, synced to wherever the repo goes.

The fix took an evening: rotate the key, move it to an environment variable, and add a check that greps every commit for credential-shaped strings. The lesson is chapter one's lesson wearing a security badge: what the machine can check, the machine should check, because you have already proven you won't.

§ 03.4

Make the AI draw it

You do not have to reconstruct the map by hand. The tool that built the accumulation can read it back. Open a session and ask:

> "Read this codebase and produce the five-box architecture map: client, server, data, storage, third parties. For every arrow between boxes, tell me what stops the wrong person from using it. Then list every place a secret lives, with file paths."

Then verify the answer against the code, arrow by arrow, the way chapter one taught you to read diffs: the AI's map is a draft, not a fact. An hour of this is the cheapest security review you will ever run. What you confirm goes into the spec's SHAPE section. What you can't confirm goes on the NOW list, because an arrow you can't explain is work, not trivia.

What you end up with is a file, not a memory. Update it the day an arrow moves.

§ 03.5

Pre-flight: the five locks

PRE-FLIGHT · ONE LOCK PER ARROW — RUN TONIGHT

- [ ] **The client enforces nothing alone.** Every rule the browser applies exists on the server too. Pick your most sensitive action and trace where it's actually checked.
- [ ] **Every query filters by owner.** Better: ownership lives in the database (row-level security), so a forgetful query returns nothing, not everything.
- [ ] **Storage is private by default.** Copy a file URL from your app, open it logged out. If it loads, that's tonight's work.
- [ ] **Secrets live in environment variables on the host.** Grep the repo for anything key-shaped; rotate whatever you find. Then make the grep a pre-release check.
- [ ] **Money truth comes from verified webhooks.** The provider's signature, checked on the server, is the only "it's paid" your app believes.

Five boxes on a napkin, one lock per arrow, drawn once and kept in the repo next to your spec. The next chapter takes this map to the place where it gets stress-tested for real: the day the app leaves your laptop.

04

Deploy day

Environment variables, databases, domains, SSL, secrets. The pre-flight list, in the order things bite you.

SUBJECT	PRODUCTION · ENVIRONMENTS	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER FOUR OF TEN
TIME	A SIX-MINUTE READ	REV	2026.08

§ 04.1

Production is a different machine

Chapter one told you about my three lead forms that said “Got it” to every visitor for weeks while delivering nothing, because one environment variable never made it to the live host. This chapter is that story’s autopsy, generalized: why the demo lies, and the exact order in which production bites.

The mental model that explains nearly every deploy-day failure fits in one sentence: your laptop is full of invisible help, and production has none of it.

YOUR LAPTOP

```
.env file with every key
logged-in CLIs
database you seeded by
hand
localhost URLs everywhere
files from six months of
work
```

PRODUCTION

```
your code

…and nothing else.
Every value you didn’t
explicitly provide is
missing.
```

Six months of building leaves your machine soaked in implicit state: keys in a local file, tools already logged in, a database you seeded by hand in week two and forgot about, URLs that say localhost. Your app doesn’t just run your code. It runs your code plus all of that, and only the code makes the trip to production.

Deploy day is one job: making every piece of invisible help explicit, on a machine that starts with nothing.

FIELD NOTE

This is also why “works on my machine” is a punchline among engineers. It is always true and never useful. The machine that matters is the one strangers reach.

§ 04.2

The order things bite you

1. Environment variables

The number-one killer, and the reason chapter one’s forms died. Your code reads keys and URLs from the environment. Locally they live in a `.env` file that never leaves your machine, which is correct. On the host, every one of them must be entered by hand, and there are usually three separate environments to enter them into: production, previews, and development.

Two traps inside the trap. First, the silent kind of missing: a well-meaning AI wraps the key check in a fallback, so instead

of crashing, the app logs a warning nobody reads and tells the user everything worked. Prefer the loud failure. A crash on deploy day beats a lie for a month.

FROM THE BUILD LOG ENTRY · 2026-08-31

Installed is not live

The morning I fixed the dead forms, I added the missing email key to the host, re-ran my test, and it was still dead. Ten minutes of confusion, then the second lesson of the day: environment variables are read when the app is built, not when you save them in the dashboard. Nothing changes until the next deploy.

The rule that stuck: env change, then redeploy, then test. Three steps, always in that order, never two.

The habit that makes this box safe: keep a committed `.env.example`, the ledger of every variable your code reads, with no values. It is the invariant list from chapter one, wearing an ops badge.

.env.example

```
# Every var the code reads. No values here, ever.
# Sends all transactional email. Sending-only key.
RESEND_API_KEY=
# Postgres connection. Prod value lives ONLY on the
host.
DATABASE_URL=
# Stripe. Webhook secret pairs with the endpoint,
ch. 6.
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
```

2. The database

Your production database is born empty, and it is not the one on your laptop. Three things bite here, in order: the connection string is an environment variable (see above, twice); your schema must arrive by migrations, not by the pile of hand-edits your local database accumulated; and whatever safety rules you built in chapter three, row-level security included, must actually be enabled in the hosted

instance. “It worked locally” often means “my local database was lawless and forgiving.”

3. Domains, SSL, redirects

Your app is live on the platform’s URL in minutes. The day feels done. Then you wire the real domain, and this box turns out to have teeth: DNS records, certificates, and the apex-versus-www question every site answers whether it knows it or not.

The rule: one of the two is the real address, the other one redirects to it, exactly one hop. Then verify all three doors: the apex, the www, and the platform URL, all serving the same build.

FROM THE BUILD LOG — ENTRY · 2026-08-31

The redirect loop I shipped this afternoon

While writing this very chapter, I made the classic move. My www domain was manually pinned to a specific deployment, which meant every deploy required me to re-point it by hand, and forgetting would serve visitors last week’s site. Fixing that properly meant attaching www to the project. I ran the command.

The platform attached www with a default redirect to the apex. The apex already redirected to www. Both public doors spun in a redirect loop, fifty hops deep, and my site was down for two minutes on a Sunday while I unwound it.

Two lessons. Redirects have two ends, and you check both directions before and after touching either. And a two-minute outage you catch yourself, on a quiet day, is the cheap tuition: the check I now run took thirty seconds to write and runs every time.

4. Public URLs and callbacks

Localhost is a private address. The moment a third party needs to reach you (a payment webhook, an email bounce, a calendar callback), your app needs a public URL, and every one of those integrations you tested locally was tested against a machine the outside world cannot see. Each has a live-mode switch and a callback URL field, and each must be

re-pointed at the real domain. Chapter 6 walks the payment version in detail, because that is the one that costs money.

5. The frozen client

One subtlety that bites late: some of your configuration is baked into the browser bundle at build time. In most frameworks, that means the variables with a public prefix. Change them and, as with every environment variable, the change means nothing until a rebuild. If a “fixed” value keeps showing up wrong, you are usually looking at a stale build, not a stubborn bug.

§ 04.3

Verify in final form

The through-line of all five boxes, and of chapter one’s dead forms: “it works” is a claim about the exact path you tested, on the machine that matters. Testing the form on localhost proves localhost. The only test that counts is the one a stranger could have run: the live site, the real form, the actual email arriving in an inbox you can open.

So deploy day ends with a ritual, not a feeling:

PRE-FLIGHT · DEPLOY DAY — RUN TONIGHT

- ☐ **List every variable the code reads,** and confirm each exists on the host for each environment.
  Your `.env.example` is the checklist; the AI can generate it by searching the code for every environment read.

- ☐ **Env change → redeploy → test.** In that order, every time. Installed is not live until the next build.

- ☐ **Migrate the hosted database and confirm its safety rules are on.** Your laptop's database is not evidence.

- ☐ **Open all three doors:** apex, www, and the platform URL. Confirm one redirect hop at most and the same build behind each.

- ☐ **Fire every integration once, for real, on the live site.** Submit each form and watch the artifact arrive: the email in the inbox, the row in the database, the webhook in the log. Ten minutes that would have saved my forms three weeks of silence.

The app is live, the doors all open, the machinery verified from the outside. Which is exactly when a different question arrives: now that strangers can reach it, what can the wrong stranger do? That is the next chapter.

05

The security pre-flight

Row-level security done right, the auth pattern that survives, and the hardcoded keys you left in. Two checks catch most of it.

SUBJECT	AUTHORIZATION · SECRETS	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER FIVE OF TEN
TIME	A FIVE-MINUTE READ	REV	2026.08

§ 05.1

Nobody is targeting you. Everything is scanning you.

The chapter-four ritual ended with strangers able to reach your app. Here is the uncomfortable arithmetic of that: within hours of your domain going live, automated scanners found it. Not hackers who care about you. Scripts that knock on every door on the internet, all day, forever, looking for the same handful of unlocked ones.

That reframing is good news. You are not defending against genius. You are defending against a checklist, which means a checklist can defend you. Solo-builder security is not a discipline you study for a year. It is two questions, asked honestly, plus the habits from chapters one through four you already have.

> **Check one:** can the wrong user read someone else's rows?
>
> **Check two:** is anything secret reachable from public?
>
> Most of what actually goes wrong in solo builds is one of these two, wearing different clothes.

§ 05.2

The distinction the AI skips

AUTHENTICATION VS. AUTHORIZATION

> Authentication is who you are: the login. Authorization is what you may do: which rows, which files, which buttons are truly yours. AI tools wire authentication well, because it comes from a library. Authorization they skip, because it comes from your product's rules, and chapter two told you where those live: in your head.

This is why the most common vulnerability in AI-built apps is embarrassingly plain: the login works perfectly, and then any logged-in user can read any other user's data by changing a number in the URL. The industry calls it an IDOR. You can call it the "everyone is admin" bug. The login was never the hard part.

The auth pattern that survives has three rules. Use the platform's auth, never your own: password storage is a solved problem and your version will be worse. Keep the session in the server-managed cookie the library gives you. And the load-bearing one: the server learns who is asking from the session, never from what the client sent. Any request that carries "userId" as an input is a request the AI wrote for the demo, not for strangers.

§ 05.3

Row-level security, done right

Chapter three called ownership-in-the-database "the rule is physics." Here is the whole recipe, concretely. It is shorter than most readers expect.

Every table gets an owner column. Row-level security gets switched on for every table, which flips the database to deny-by-default: a query with no matching policy returns nothing. Then one policy per access pattern, and for a solo SaaS there is usually just one that matters:

migration: lock clients to their trainer

```
alter table clients enable row level security;
create policy "trainers see only their own clients"
on clients for all
using (trainer_id = auth.uid());
-- auth.uid() = the signed-in user. The
-- helper's name varies by database host.
```

Ten lines like these, and the "everyone is admin" bug becomes structurally impossible: a forgetful query returns an empty list instead of the whole customer base. The AI can rewrite your queries badly forever and the database keeps the rule, because the rule no longer lives in the queries.

Two keys come with this setup, and they are opposites. The anonymous key ships to the browser and is safe to be public, precisely because the policies stand behind it. The service key bypasses every policy and belongs only in the server's environment variables, chapter four style. Mixing those two up is the one way to undo everything above.

And the test, because chapter one taught you that rules you don't check are rules you don't have: **the two-account test.**

FIELD NOTE

Ordani, the HIPAA-compliant app from these pages, runs its entire authorization model this way: a birth worker sees her clients and nobody else's, enforced in the database. Both outside security reviews walked those policies line by line.

Create data as user A. Log in as user B and try to read it: through the app, and through the lazy door, by editing IDs in the URL and in API calls. Five minutes. If B sees anything of A's, you have tonight's work, and you found it before a stranger's script did.

§ 05.4

The keys you left in

Check two is about secrets, and there are exactly three places they hide in an AI-built repo.

In the code and config. Chapter three's build-log entry found one live token pasted twelve times by helpful sessions. The grep-for-credential-shapes habit covers this, if you actually run it.

In the browser bundle. Framework variables with a public prefix ship to every visitor. The test is view-source honesty: open the deployed site, view the page source and the JavaScript it loads, and search for `sk_`, `secret`, `key`. If a privileged key appears, every visitor already has it.

In the history. Git remembers. A key committed once and deleted later is still in the repository's past, and anything that has ever synced to another machine has copies. Which is why the rule for a leaked key is never "delete it." It is rotate it: issue a new one, kill the old one, and the history can keep its souvenir.

FROM THE BUILD LOG

ENTRY · 2026-08-31

The secret I leaked to be helpful

This one is mine, from this week. Moving fast on my own site, I pasted two live credentials, an email API key and a private calendar URL, straight into an AI chat session to get a feature wired. It worked. The feature shipped that morning.

And the moment they hit the chat, both stopped being secrets. Not because anyone malicious was watching, but because they had touched a surface I don’t control, and “probably fine” is not a security model. Rotation went on that day’s list, with a deadline, written where I can’t ignore it, before the feature was even finished.

If reading this just reminded you of a key you pasted somewhere once, congratulations: you have found your rotation list. The question is never “did it leak?” It is “can I afford to assume it didn’t?”

Pre-flight: the five checks

PRE-FLIGHT · SECURITY — **RUN TONIGHT**

- ☐ **Run the two-account test.** Create as A, snoop as B, through the app and through edited IDs. Anything visible is tonight's work.
- ☐ **Row-level security on, on every table.** Deny by default, one policy per pattern, service key server-side only.
- ☐ **View-source the deployed site.** Search the page and its scripts for anything key-shaped. The browser bundle is public forever.
- ☐ **Grep the repo and its history for credential shapes.** Anything found gets rotated, never just deleted.
- ☐ **Hit your admin routes logged out.** Cold, in an incognito window. Hidden buttons are not authorization, and scanners don't use buttons.

Two questions, five checks, one evening. That is most of solo-builder security, and it is more than the scanners are betting you did. The next chapter follows the money: the payment stack, where the stakes stop being embarrassment and start being refunds.

06

Stripe in production

Webhook reliability, refunds, subscription edge cases, and the test-to-live failures nobody warns you about.

SUBJECT	PAYMENTS · WEBHOOKS	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER SIX OF TEN
TIME	A SIX-MINUTE READ	REV	2026.08

§ 06.1

The stakes change here

Every failure in this book so far cost you embarrassment or a weekend. This chapter's failures cost money and trust, in public, one customer at a time. A broken upload annoys someone. A customer who paid and got nothing tells everyone.

The good news is the same shape as chapter five's: you are not inventing payment security. Stripe carries the hard parts, and your job reduces to three sentences.

> Your app never touches money. It sends people to Stripe, believes only what Stripe's signed webhooks say happened, and keeps its own "paid" column as a mirror of Stripe's records. Every payment bug in solo builds is a violation of one of those three sentences.

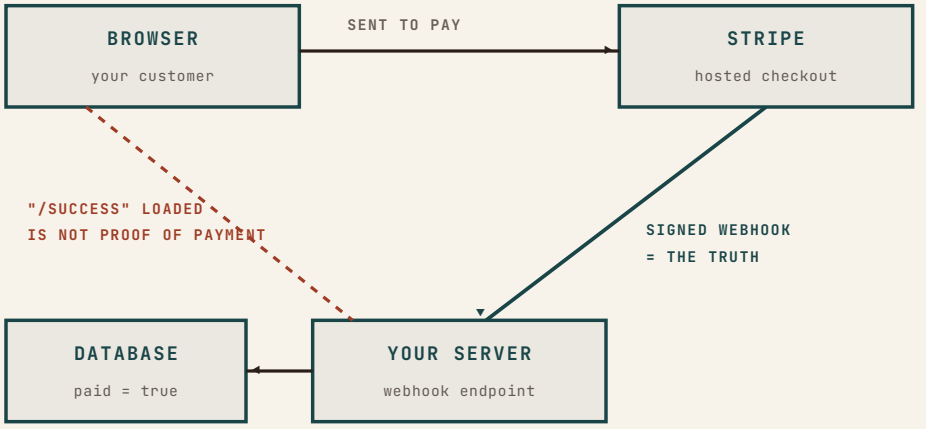
§ 06.2

The flow that survives

Use Stripe's hosted checkout page, not a card form of your own. The AI will happily build you a beautiful custom card form, and it will be a beautiful liability: card data routed through your code drags you into compliance territory an entire team would respect, and conversion is worse than the page buyers already recognize. Send them to Stripe; Stripe sends them back.

SOURCE OF TRUTH

> For money state, Stripe's records are the fact and your database is a cache of the fact. When the two disagree, Stripe is right, and your job is to resync, not to argue.



MONEY STATE FLOWS ONE WAY: STRIPE, THEN SERVER, THEN DB

The diagram has one villain, and it is the dashed line: the success page. When Stripe redirects your customer to `/success`, that page load proves navigation, not payment. Tabs close. Networks drop. And anyone can type `/success` into a URL bar. The single most common money bug in AI-built apps is granting access on that page load, because it demos perfectly and fails in production in both directions: paying customers who closed the tab get nothing, and curious visitors who typed the URL get everything.

Access is granted in exactly one place: the webhook handler.

§ 06.3

Webhooks, the load-bearing wall

A webhook is Stripe calling your server: an HTTP POST to an endpoint you registered, carrying “checkout completed” or “charge refunded.” Chapter three drew this arrow with the label “verified webhooks,” and now it earns its keep. Four rules make the wall hold.

Verify the signature, every time. Your endpoint is a public URL. Without the signature check, anyone on earth can POST “payment succeeded” to it and grant themselves your product. Stripe signs every event with a webhook secret; the check is three lines with the SDK, and the secret is an environment variable, chapter four style.

Welcome retries; process events once. Stripe retries delivery until you answer with a 200, which means the same event can arrive twice. Record processed event IDs and skip

repeats, or your database will double-grant, double-log, and double-email.

Trust the event, not the order. Events can arrive out of sequence. Handle each one by asking Stripe's records what the current state is, rather than assuming the last event you saw told the whole story.

Answer fast, work after. Acknowledge the event, then do the slow parts. An endpoint that dawdles gets retried, and now you are back to rule two. "After" needs no clever machinery at solo scale: save the event to a table, answer, and let a job that runs every minute send the emails and do the slow writes.

Hand the four rules to the tool rather than remembering them at three in the morning. This prompt ships in the companion pack:

prompts/stripe-wiring.md

```
Implement checkout using the provider's HOSTED
checkout page, never a custom card form, and a
webhook endpoint that follows all four rules:
1. Verify the webhook signature on every event;
   reject on failure.
2. Record processed event IDs and skip duplicates.
3. On each event, read current state from the
   provider's API rather than trusting event order.
4. Acknowledge fast; do slow work after responding.
Access to the product is granted ONLY in the
webhook handler, never on the success page. Add
refund handling that revokes what payment granted.
List every environment variable this needs, and
which are per-mode.
```

§ 06.4

Test-to-live: the five silent swaps

Stripe's test mode is a parallel universe: same dashboard, same API shape, fake money. The day you flip to live, five things silently do not come along, and each one fails without an error message.

1. **The keys.** `sk_test_` and `sk_live_` both work perfectly, each in its own universe. Live site with test keys means

FIELD NOTE

Local development wrinkle: Stripe cannot call localhost. The Stripe CLI forwards live test events to your machine while you build. It also hands you a different signing secret than production uses, which is the next section's trap in miniature.

imaginary revenue; a test script with live keys means real charges.

2. **The webhook endpoint.** Registered separately per mode. Your test webhook does not fire for live events; live needs its own registration.
3. **The webhook secret.** The new live endpoint gets a new signing secret, and the old one keeps verifying test events only. Signature failures after go-live are this, almost every time.
4. **The prices.** Products and price IDs are per mode. The `price_...` your checkout references in test does not exist in live until you recreate it.
5. **The redeploy.** All four above live in environment variables and configuration, and chapter four already taught the rule: installed is not live. Change, deploy, then test.

§ 06.5

Refunds, disputes, subscriptions

Refunds at solo scale are a dashboard click, and that is fine. The part that belongs in your code is the echo: the click fires a `charge.refunded` webhook, and your handler revokes what the payment granted. A refund policy your code doesn't hear about is a discount, not a policy.

Disputes are the one email you never ignore. When a cardholder disputes a charge, silence is an automatic loss. Submit the evidence Stripe asks for: what was bought, when it was delivered, the logs that show usage. Solo builders lose most disputes they contest and all of the ones they don't; the difference funds the habit.

Subscriptions add exactly three edge cases worth engineering for on day one. Failed renewals: cards expire, and Stripe retries on a schedule, so treat `past_due` as a grace period with a friendly email, not an instant cutoff. Cancellations: "cancel at period end" is almost always what your customer means; immediate cancellation with no refund is how you earn the dispute above. And status: your app reads subscription state from one place, the column your webhooks maintain, never from assumptions about what the customer probably did.

FROM THE BUILD LOG

ENTRY · 2026-08

The buy button I refused to ship

The sales page for this manual went live weeks before its checkout existed. The temptation was strong to ship the $149 button anyway, wired to nothing or to something untested, because a page without a buy button feels unfinished.

It shipped as a waitlist instead: leave an email, get chapter one free, be told the day it opens. The reasoning is this chapter in one sentence: money UI ships last, after the money path is verified end to end, because the exact moment someone decides to pay you is the worst possible moment to show them a lie.

The first real charge on any build should be your own card, at the live URL, watched all the way through: checkout, webhook, database flip, confirmation email. Then refund yourself and watch the refund echo through the same pipe. That is the two-account test of money, and it costs you nothing but the fee.

Pre-flight: money

PRE-FLIGHT · STRIPE — **RUN TONIGHT**

- [ ] **Hosted checkout only.** Your server never sees a card number, and the AI's beautiful custom card form stays in the parking lot.
- [ ] **Access is granted by the verified webhook, never by the success page.** Signature checked, event IDs deduped, retries welcomed.
- [ ] **Make the five live-mode swaps deliberately:** live keys, live endpoint, its new secret, live price IDs, then redeploy. Each is an env change, and installed is not live.
- [ ] **Wire the refund echo.** `charge.refunded` revokes what payment granted. Test it with a real refund to yourself.
- [ ] **Pay yourself once, live, and watch the whole pipe:** checkout, webhook, database, email. Then refund it. The fee is the cheapest audit you will ever buy.

Money in, money safe, money honest. What's left is the paperwork the outside world attaches to it, and when that paperwork actually applies to you: HIPAA, SOC 2, GDPR, and the acronyms in between.

07

Compliance, when it matters

HIPAA, SOC 2, GDPR. When you genuinely need them, when you don't, and what compliant actually requires.

SUBJECT	REGULATION · TRUST PAPERWORK	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER SEVEN OF TEN
TIME	A SIX-MINUTE READ	REV	2026.08

§ 07.1

The two ways builders get this wrong

Compliance breaks solo builders in opposite directions. Half ignore it entirely and ship health data through consumer tooling with a shrug. The other half freeze: they have a genuinely good idea near a regulated space and never build it, because the acronyms feel like a wall only companies with lawyers climb.

Both failure modes come from the same misunderstanding: that compliance is a mysterious purchased credential. It mostly is not. It is the chapter-three map and the chapter-five locks, with paperwork attached, applied where the law actually reaches you. I built a HIPAA-compliant app alone. Not because I am a lawyer, but because the requirements, read calmly, turned out to be a list, and lists are what this manual does.

FIELD NOTE

Honesty first: this chapter is a field guide from a builder, not legal advice. When real money or real patient data is on the line, one hour with a healthcare or privacy attorney is cheap insurance, and this chapter makes that hour ten times more productive.

§ 07.2

Which of these actually applies to you

Three acronyms cover most of what a solo SaaS will ever meet. Each has a one-question test.

Do you store or move **identifiable health data** on behalf of providers or plans?	**HIPAA · NOW** BAAs with every vendor touching that data, access controls, audit trail, breach plan.
Can **EU or UK residents** sign up for your public app?	**GDPR BASICS · NOW** Honest privacy policy, data export and deletion on request, vendor DPAs.
Is an **enterprise buyer's questionnaire** blocking a real deal?	**SOC 2 · WHEN ASKED** An audit report you buy when revenue justifies it. Until then: a one-page security summary.

HIPAA is United States health-data law, and the test is precise: it reaches you when you store or transmit individually identifiable health information on behalf of providers, plans, or their partners. A meditation app with self-reported moods is usually outside it. A tool where clinicians, doulas, or therapists manage client records is inside it, and inside means the full lane above.

GDPR (and its cousins, like California's CCPA) is about personal data generally, and its test is almost a formality: if EU residents can sign up, assume it applies. The solo-scale core is smaller than its reputation: tell the truth about what you collect, be able to hand a user their data, be able to delete it, and have agreements with the vendors who process it for you.

SOC 2 is the odd one out: not a law at all. It is an audit report enterprises request during procurement, a trust document with an invoice attached, typically five figures a year once auditors and tooling are in. The test is brutally practical: is a real deal, with real revenue, blocked by a questionnaire demanding it? If not, do not buy it on spec. A one-page

security summary describing your chapter-five practices answers most questionnaires a solo builder will see.

§ 07.3

What “compliant” actually requires

Strip the acronyms and the same skeleton sits under all three. You already met most of it.

Know where the data lives: the five-box map from chapter three, drawn and current. Control who sees it: chapter five’s row-level security and the two-account test, which double as HIPAA’s “access controls” in a form an auditor can read. Encrypt in transit and at rest, which modern platforms do by default; your job is to verify, not to build. Keep an audit trail of who touched what. Be able to export and delete a person’s data on request. Put agreements in place with every vendor that touches the data. Write down what you do, and then do what you wrote down.

That last sentence is the entire spirit of the thing. Compliance failures are rarely exotic. They are gaps between the policy and the practice, and a solo builder has one advantage no enterprise has: the policy and the practice live in the same head.

Compliance is the napkin, plus the locks, plus paperwork. You built the first two chapters ago.

§ 07.4

The vendor chain is the real exam

Here is the part that decides most of it at solo scale: you inherit compliance through your vendors. Your app is as compliant as the weakest agreement in the chain.

Under HIPAA the agreement is a **BAA**, a business associate agreement, and every vendor that touches identifiable health data must sign one: your hosting, your database, your file storage, your email sender, your error logger. The big platforms sign them, some only on certain tiers. Plenty of

consumer-grade tools never will, and one convenient no-BAA tool in the chain, one crash-reporting service that captures a record, one email API relaying an appointment reminder, quietly breaks the whole thing.

Under GDPR the same role is played by the **DPA**, the data-processing agreement, and the mainstream vendors bundle one into their standard terms. Your work is mostly to collect and file them, and to notice the vendor that has none.

So the exam is an afternoon, not a year: take the chapter-three map, list every vendor on it, and put a checkmark or an X next to each. The X's are your work: replace the vendor, upgrade the tier, or keep that category of data away from it.

FROM THE BUILD LOG — ENTRY · 2026-08-30

The stack I stopped naming

While polishing the public case study about my own health app, a review caught something I had walked past: the page proudly named the exact infrastructure the app runs on, vendor by vendor.

Read as marketing, it was credibility. Read as an attacker, it was a map: knowing precisely which platforms power a health app tells you which known vulnerabilities to try, which login pages to spray, which status pages to watch for a bad week.

Every vendor name came off the page the same day. The architecture napkin is for you, your spec, and your auditors. The public version says what the system does and what protects it, never which brands it is assembled from. Confidence talks about properties. Only carelessness publishes the parts list.

FIELD NOTE

Ordani, the HIPAA-compliant app from these pages, came down to exactly this: a vendor chain where everything touching client data carries the right agreement, ownership enforced in the database, and the two-account discipline from chapter five. Two outside security reviews later, the design holds.

Pre-flight: the paperwork pass

PRE-FLIGHT · COMPLIANCE — RUN TONIGHT

- [ ] **Answer the three questions** in the diagram, in writing, in your spec's NOT or NOW section: health data for providers, EU users, enterprise questionnaires. Most readers get one yes, not three.
- [ ] **Run the vendor sweep.** Every vendor on your chapter-three map gets a DPA checkmark, plus a BAA checkmark if health data is in play. Any X becomes a replacement or an upgrade, this week.
- [ ] **Write the honest privacy policy:** what you collect, why, how long you keep it, and how a user gets it exported or deleted. Plain language beats borrowed legalese you don't actually do.
- [ ] **Make deletion a query, and prove it.** On a test account, delete one user's everything: rows, files, logs that identify them. Chapter three's owner column is why this is an evening, not a quarter.
- [ ] **Hold the SOC 2 line.** Keep a one-page security summary for questionnaires, and buy the audit the day a real contract makes it worth it, not the day the anxiety does.

The data is mapped, locked, and papered. The app deserves to exist in public. Which raises the question this book has been saving: where do the first ten people who pay for it actually come from?

08

The first ten users

Getting to the first ten people who keep using it. Where they come from, and why posting stopped working.

SUBJECT	DISTRIBUTION · FIRST USERS	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER EIGHT OF TEN
TIME	A SIX-MINUTE READ	REV	2026.08

§ 08.1

It shipped. Nobody came.

The sales page of this manual opens with three sentences of pain, and this chapter is the third one. The app works. The deploy held. The security checks passed. You posted it where builders post things, refreshed the analytics, and watched a spike of visitors arrive, click twice, and never return.

Here is the part that should comfort you, and then the part that should change your week. The comfort: this is not evidence your product is bad. The launch-post lottery was always a lottery, and it has gotten worse, because the feeds are now full of AI-built demos that all look competent. A working app is no longer rare enough to be news. The change: you were never actually playing for a crowd. You were playing for ten people, and ten people have never been reachable by broadcast.

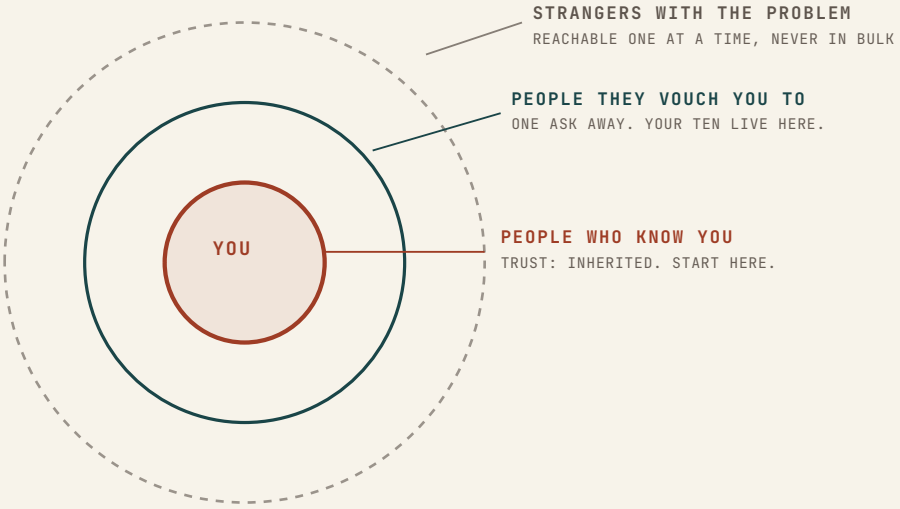
A USER, VERSUS A TOURIST

> A tourist clicks, pokes, and leaves. A user returns without being reminded. Your first milestone is ten users, by that definition, and nothing on an analytics dashboard counts toward it.

Selling has been my trade longer than building: enterprise software inside SurveyMonkey on the way to its IPO, and $20M+ in client revenue since. Every expensive thing I know about finding the first customers compresses into one sentence: the first ten come from conversations, not audiences.

§ 08.2

Where the ten actually live



BROADCAST REACHES NONE OF THESE RINGS. CONVERSATIONS REACH ALL THREE.

The rings are ordered by trust, and trust is the whole game at zero users, because a stranger trying an unknown app is spending the scarcest thing they have: the willingness to be disappointed again.

The inner ring is people who already know you. Not "my followers." The forty people who would answer a text. Most builders skip this ring out of embarrassment, which is exactly backwards: these are the only people on earth who will forgive a rough edge because they trust the person behind it.

The move in this ring is not "check out my app." It is one message, one person, no blast: name the specific problem, ask if it is real for them, and if it is not, ask the ring-two question: "who do you know who deals with this?" A no that produces an introduction is a better outcome than a pity signup.

The second ring is the people they vouch you to. An introduction moves trust you did not have to earn. This is where most of your ten live, and the arithmetic is humbling and freeing at once: ten users at a plausible hit rate is somewhere around a hundred conversations. A hundred conversations is a month of mornings. No audience required.

The outer ring is strangers who verifiably have the problem: the forum where they complain about it, the professional group where they gather, the community your niche actually

uses. You reach them the same way, one at a time, by being useful in their space before you ask for anything. The niche matters more than the platform: birth workers, to pick an example I know, are not on the builder feeds at all, and no launch post would ever have found them.

§ 08.3

The message that gets a reply

Cold or warm, the anatomy of outreach that works has not changed in twenty years of selling. Four parts, in order.

Context that is really about them. One line proving this is not a blast: the mutual person, the post of theirs you read, the specific thing about their situation. If the first sentence could be sent to anyone, it will be answered by no one.

Proof you looked. Quote their own words back: the problem as they described it, in their phrasing, from their forum post or their website. Nothing earns a reply like being genuinely heard.

One concrete offer, work-shaped. Not “would love your feedback.” Offer to do something for them: set it up for them, migrate their data, run their first week yourself. At ten-users scale, unscalable is not a compromise. It is the strategy.

A one-step ask. Reply to this email. Pick a time. Send the file. One verb, low friction, no account creation standing between them and yes.

FIELD NOTE

Ordani opened as a private beta with fourteen practices, found through the rings, not through a launch. Hundreds of birth workers pay for it today, and at the six-month mark none had been lost to a competitor. Retention was the plan, not the reward.

FROM THE BUILD LOG

ENTRY · 2026-08-31

One email, not a launch

This week I needed a new client for my consulting practice, which is the same problem as first users wearing a collar. I wrote one email, to one person, introduced through a mutual friend.

It opened with her week, not my services. It quoted the best sentence from her own website back to her, because I had actually read it. The offer was work, not talk: take a week of what her current provider produces, remake all of it in my style, free, judged side by side. The ask was one step: reply.

No launch, no post, no audience. One researched email to one right person, with an offer that costs me effort instead of costing her trust. That shape closed enterprise deals when I carried a quota, and it is exactly the shape that finds ten users.

One more thing about the writing itself, because this manual is about building with AI: let the model draft the paragraph, never the specifics. The personal layer, the mutual friend, the quoted sentence, the offer only you can make, is chapter two’s lesson wearing a sales hat: the part the AI cannot generate is the part that works. Everyone can smell machine-written outreach now. The specifics are the moat.

§ 08.4

Track ten people like you track invariants

You will not hold a hundred conversations in your head. Chapter one’s answer applies to humans too: memory goes in a file, in the repo.

USERS.md

```
# One block per person. Updated after every
conversation.
## Dana R. — ring 2, via Marcus
Problem: schedules clients in three apps, hates all
three.
Last: 08-28, demo call. Asked twice about calendar
sync.
Next: onboard her real client list myself, Thu.
Returned unprompted: yes, twice. Counts toward the
ten.
```

Ten blocks like this outperform any CRM you would configure and abandon. And the file does double duty: “asked twice about calendar sync” is tomorrow’s spec input. Your first ten users are also your first ten product decisions, which is why collecting them by hand, slowly, is not the inefficient version of growth. It is the version where you learn what you built.

Pre-flight: the first ten

PRE-FLIGHT · FIRST USERS — RUN TONIGHT

- ☐ **Write the ten-name list tonight.** Inner ring first: people who would answer a text, who touch the problem or know someone who does. The list is usually easier to write than it felt.
- ☐ **One conversation a day.** Personal, specific, no blast. A no that ends in “but talk to…” is a win; log the introduction.
- ☐ **Make one unscalable offer.** Set it up for them, migrate the data, run the first week. Effort you spend is trust they don’t have to.
- ☐ **Keep USERS.md current.** One block per person, updated after every conversation. What they ask about twice goes to the spec.
- ☐ **Count returns, not signups.** Someone who comes back unprompted, twice, is one of your ten. Ten of those, and the next chapter’s loop has fuel.

Ten people who return without being reminded, each one known by name, each one a conversation you can quote. That is not a small start on distribution. That is distribution, working. The next chapter is about the loop that turns those ten into a hundred without ever touching broadcast.

09

The distribution loop

Turning the first ten into the next hundred. Reply, don't broadcast. The metric that matters before MRR.

SUBJECT	GROWTH LOOPS · REFERRALS	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER NINE OF TEN
TIME	A SIX-MINUTE READ	REV	2026.08

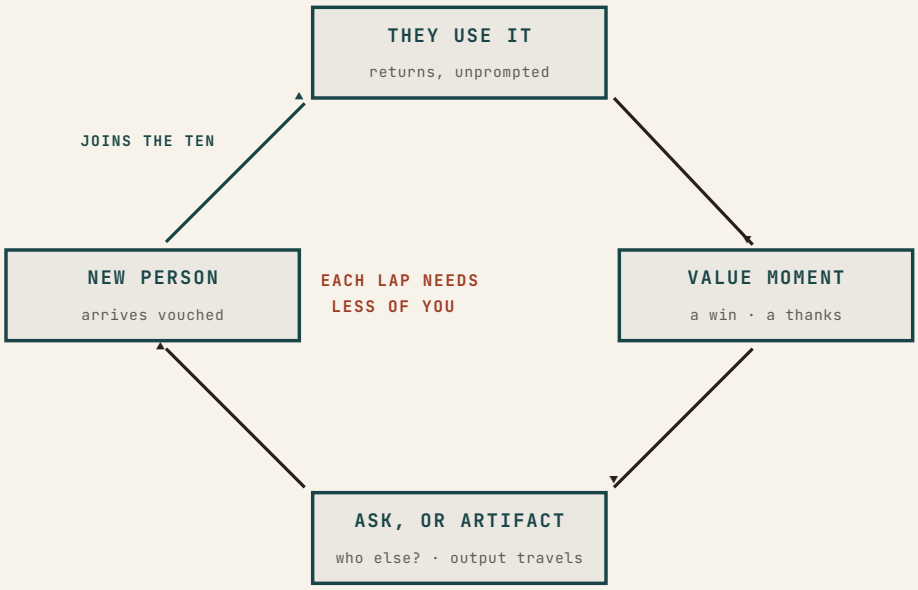
The instinct to resist

You have ten users who return without being reminded. You know their names. And right on schedule, the old instinct comes back: now we scale. Now comes the content calendar, the newsletter, maybe some ads. Now we broadcast.

Resist it. Broadcasting did not find your first ten, and it will not find your next ninety. What finds them is the same machinery from chapter eight, upgraded in one specific way: it has to start running without you pushing every lap.

DISTRIBUTION LOOP

> A repeatable path from a happy moment inside your product to a new person hearing about it from someone they trust, where each cycle costs you less effort than the last. Growth without a loop is just your labor, invoiced weekly, forever.



The loop has four stations, and you built three of them in the last chapter. Users return. Value moments happen. The new station, the one this chapter installs, is the third box: the moment where a win becomes either an ask or an artifact, and someone new hears about you from a voice that is not yours.

§ 09.2

The metric that matters before MRR

Revenue is a lagging indicator at this stage, and it will lie to you in both directions: a burst of launch-week charges from tourists, or honest zero while ten serious users evaluate. The number that tells the truth is this one:

How many of your users came from your users? Count the second-hand users. That number is your distribution, and before MRR, it is the only growth number that isn't noise.

A second-hand user arrived because a user mentioned you, sent an artifact, or made an introduction, with no push from you. Track it weekly. If it is zero, nothing is wrong with your product, but nothing is compounding either: every user you have, you personally hauled in, and every user you ever get will cost the same labor. The rest of this chapter exists to move that number off zero.

FIELD NOTE

This is also the honest read on chapter eight's Ordani numbers. Retention is what makes a loop possible at all: nobody refers a tool they are halfway out the door on.

§ 09.3

Reply, don't broadcast

Attention arrives at a small product constantly, in tiny amounts: a support email, a question in a community, a reply to your chapter-eight outreach, someone's offhand mention. Broadcasting ignores all of it to go shout at strangers. Replying compounds it.

The doctrine is one habit with two venues. In private: answer every user email fast and personally, because at your scale, support is marketing, and the person who got a founder's reply in nine minutes tells that story for you. In public: answer real questions where your users gather, the forum thread, the professional group, the subreddit, completely and generously, without pitching. A good public answer is a

permanent artifact: it keeps being found, keeps being useful, and keeps carrying your name to ring three long after you wrote it.

That is also the honest way to think about “content.” A content calendar asks what should I say this week? The reply doctrine asks what did someone actually ask? The second question has an inexhaustible supply and a guaranteed audience of at least one.

§ 09.4

Engineer the ask, then the artifact

The ask. Chapter eight’s ring-two question, “who do you know who deals with this?”, now gets a schedule: it fires at the value moment, every time. Someone says thanks. Someone hits a win. Someone renews. That is the one moment when helping you feels like the natural next sentence, and the ask must be specific to be answerable: not “tell your friends,” but “which other trainer do you know who’s still juggling three apps?” One name is a win. Log the introduction in USERS.md and run chapter eight on it.

The artifact. The stronger station, because it works while you sleep: make something your product produces in normal use visible to people who don’t use it. The booking page a trainer sends every client. The invoice with quiet, dignified product credit in the footer. The export, the report, the shared link. When the product’s output travels to non-users as a matter of course, every active user is distribution, and nobody had to be asked anything.

The test for a good artifact is that it serves the recipient first: the client got a booking page that worked beautifully on her phone. The credit line rides along; it never leads.

FROM THE BUILD LOG

ENTRY · 2026-08-31

The book that carries its own loop

You are holding this chapter's example. The first chapter of this manual is free, and it ships with its loop built in: the copyright line reads "share it, don't sell it," because a shared chapter is the artifact doing its job. Every page you are reading carries the URL in the footer, so the artifact knows the way home. And the email that delivers chapter one ends with an ask sized to a value moment: hit reply if it lands. I read every response.

None of that is growth hacking. It is one free artifact built to travel, one specific ask at the moment the reader has gotten something, and replies answered by a person. The same three moves this chapter just handed you, running on the book that taught you them.

§ 09.5

What not to build yet

Three things wait until the loop runs. **Paid ads**: buying traffic without a loop is pouring water into a bucket you haven't checked for a bottom; ads amplify loops, they do not create them. **SEO at scale**: a library of articles for strangers can wait until the replies you write in public tell you which questions are worth a permanent page. **The growth newsletter**: a broadcast list of strangers is a vanity number. The list worth mailing is the one chapter six's waitlist built: people who asked to hear from you, written to like humans, rarely.

LOOP.md — weekly, five lines

```
# Week of 09-01
Returning users: 11 (+1)
Conversations: 6 · Asks made: 4 · Intros received:
2
Second-hand users: 1 (Dana sent her colleague.
FIRST ONE.)
Artifact check: booking pages viewed by 31 non-
users this week.
```

Five lines, once a week, next to USERS.md and your spec. The week the second-hand line stops reading zero is the week you have a business instead of a project, whatever the revenue says.

§ 09.6

Pre-flight: the loop

PRE-FLIGHT · DISTRIBUTION — RUN TONIGHT

- [ ] **Count your second-hand users, today.** Users who came from users. Zero is normal and temporary; it is also the number everything below exists to move.
- [ ] **Fire the ask at every value moment.** A thanks, a win, a renewal: one specific who-else question. One name is a win; log it and run chapter eight on it.
- [ ] **Ship one traveling artifact.** Find the output non-users already see, or should see, and make it excellent for the recipient, with a quiet credit line that knows the way home.
- [ ] **Reply to everything within a day.** Privately to users, publicly where they gather. Support is marketing at your scale, and public answers compound.
- [ ] **Keep LOOP.md, five lines a week.** Ads, SEO, and newsletters wait until the loop runs without you pushing every lap.

Ten became the machinery for a hundred, and the machinery runs on retention, replies, and artifacts instead of your mornings. Which surfaces the last question this book owes you, the one nobody builds alone forever without asking: when is the right time to stop being the only pair of hands?

10

When to hand it off

The signals you've outgrown solo. When to hire, when to rent senior help, when to sell, and when to keep going.

SUBJECT	SCALE · SUCCESSION	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER TEN OF TEN
TIME	A SIX-MINUTE READ	REV	2026.08

§ 10.1

The question arrives disguised

Nobody wakes up and asks “have I outgrown solo?” The question shows up wearing work clothes: support answered at midnight again. A feature you know exactly how to build, waiting three weeks for your hours. LOOP.md logging introductions you never followed up, because the loop now produces more than one person can catch.

Chapter one’s wall was the tool’s limit: a context window your codebase outgrew. This last wall is yours: hours, skills, and appetite, and it deserves the same treatment this book gave the first one. Not vibes. Signals, read honestly, and a decision made on purpose.

One reframe before the signals. Solo was never an identity. It was a strategy for a phase, and this manual’s whole premise is that the phase now stretches further than it ever has: one person with these tools and these disciplines reaches what took a team five years ago. The ceiling is higher. It is still a ceiling.

§ 10.2

Five signals, checkable

The queue signal. Paying users routinely wait more than a week for fixes you already know how to make. Not hard problems. Queued problems.

The ceiling signal. Growth is flat for a quarter, and the constraint is provably not demand: the loop ledger shows conversations you declined, intros gone stale, a waitlist aging.

The skill signal. The next milestone needs a craft you do not have and do not want: an enterprise sales motion, a mobile app, a compliance audit at a tier beyond chapter seven.

The dread signal. Some part of the work you now actively avoid. Dread compounds like drift: quietly, then suddenly, and it is the honest early warning of the burnout that ends projects.

The bus-factor signal. Real customers depend on a system only you can operate. You are the single point of failure in your own five-box diagram, and sophisticated customers and buyers can see it from the outside.

None of these alone means hire tomorrow. Two or three, persisting across a quarter, mean the question is live and deserves a written answer.

§ 10.3

The four roads

The signals are absent, margins are strong, and the ceiling is still above you.	**KEEP GOING** Legitimate, and further-reaching than ever. The one rule: choose it on purpose, in writing.
The overflow is **repeatable and full-time shaped**: support, success, operations, content.	**HIRE** Hire against the dread list. Write the role as rules-with-reasons first.
The gap is **senior judgment, for a season**: a launch, an enterprise motion, a compliance push.	**RENT IT** A fractional operator: days a week, months not years. Define the milestone first.
The appetite is gone, or someone strategic values it more than you do.	**SELL** A real market for small, profitable software. Sell from strength, never the burnout bottom.

Three of the four deserve a closer look, and one deserves a disclosure.

Hiring goes wrong in one predictable way for solo builders: the first hire is a junior developer “to help with the code,” and now you run two jobs, the product and a mentorship, while the code review from chapter one doubles. The work that actually transfers first is the repeatable kind. And before any hire, run this book’s reflex: write the job as rules-with-reasons, and if a file plus a machine check can do it, ship that instead of a salary.

Renting is my own trade, so weigh my bias as you read this. The honest test survives the bias: if the gap is senior judgment for a season, renting beats hiring, because milestone-shaped needs end and salaries do not. If the gap is repeatable hours forever, renting is an expensive way to avoid a decision. Scope it like an engagement, not a marriage: the milestone, the season, what done means, written before the search starts.

Selling has a fact most solo builders never hear: everything this manual made you write down is your data room. The spec, the invariants, the architecture napkin, the vendor sweep, the machine-run checks, USERS.md, LOOP.md: a buyer's diligence is a walk through files you already keep, and a loop that runs without your push (chapter nine's whole point) is precisely what gets paid for. You have been building the exit packet since chapter one, whether or not you ever use it.

FROM THE BUILD LOG

ENTRY · 2026-08-31

Written the week I chose road one

Full disclosure about the person giving this advice: I am writing this final chapter in a week where hundreds of people pay for an app I still run alone, next to a consulting practice. The signals get scored, and I keep choosing solo, on purpose, on terms this book taught me.

The terms are the point. Every rule a machine can run, runs by machine. Memory lives in files, so no session, human or otherwise, starts from amnesia. The loop moves on artifacts while I sleep. Solo stopped meaning "everything depends on me" the day the systems started carrying their share, and that is the only version of solo I would recommend to anyone.

The day the signals flip, I will take this chapter's advice like anyone else. It is written to the same standard as the rest of the book: advice I am willing to follow in public.

FIELD NOTE

The exits I was inside all taught the same pattern: what gets valued is what keeps working when the hero takes a vacation. The list is on the last page.

The wall, one last time

Look at what this book actually taught, end to end. Chapter one: the tool's memory fails, so judgment moves into files. The spec carried your intent. The database carried your rules. Stripe carried the money truth. The checks carried your hard-won lessons. The artifacts carried your product to strangers. Every chapter was the same move at a different altitude: take something that lived only in you, and hand it to a system that does not forget, does not sleep, and does not need you standing there.

Handing off was never the last chapter. It was the skill the whole book was teaching.

You were never learning to build alone. You were learning to build something that survives you, and that skill works at every size: a session, a launch, a company.

The last pre-flight

PRE-FLIGHT · THE HAND-OFF — RUN TONIGHT

- [ ] **Score the five signals quarterly, in writing.** Queue, ceiling, skill, dread, bus-factor. Two or more, persisting, means the question is live.
- [ ] **Choose on purpose.** "Keep going" is a decision with a date on it, recorded in the spec's NOW, not a default you drift into.
- [ ] **Before any hire, write the job as rules-with-reasons.** If a file and a check can do it, ship those instead. What remains is the real job.
- [ ] **Rent senior judgment by the milestone.** The season, the outcome, and what done means, defined before the search, reviewed at the end like any engagement.
- [ ] **Keep the repo sale-ready even if you never sell.** The files this book had you write are the data room, and a business legible to a stranger is also one that is easier to run yourself.

That is the manual. Ten chapters, one move: past the wall, on purpose, with systems that remember. Go ship the thing.

END · THE 80% WALL

Build past the wall.

Thank you for reading. The companion files that ship with this manual, the prompt files, the pre-flight checklists, and the spec templates, are the working versions of everything you just read. They arrived alongside this PDF with your purchase. Chapter one is free for anyone at micahjonesconsulting.com/playbook: send the builder you know who is stuck at 80%.

WHO WROTE THIS

I'm Micah Jones. I sold enterprise software inside SurveyMonkey on the way to its IPO, and I was inside Postmates, Guardicore, and Neuton.AI for their exits: $5B+ in combined value. My consulting work has produced $20M+ in client revenue, and Ordani, the app in these pages, is a HIPAA-compliant SaaS I built alone on the same AI tools this manual is about. Hundreds of birth workers pay for it today.

IF YOUR BUILD NEEDS A SECOND PAIR OF HANDS

Some builds need the fractional chapter of this book made real. I take a small number of engagements: the strategy and the software, from one person. Thirty minutes, bring the problem.

micahjonesconsulting.com/book